

מס' שאלון - 532 21
ביולי 2025

סמסטר 2025ב

20905 / 4

מס' מועד 91

שאלון בחינת גמר
20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 9 עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:

* שאלה 3 היא שאלה מפוצלת ל2 גרסאות :
אחת מיועדת לסטודנטים שלמדו את הקורס בסמסטר 2024ב,
והשניה מיועדת לכל שאר הסטודנטים.
לכן יש להקפיד לענות על השאלה בגרסה הנכונה לפי הסמסטר
בו למדתם את הקורס.
סטודנט שיענה על גרסה לא נכונה של שאלה זו - לא יזכה לנקודות!!!

את תשובותיכם רשמו אך ורק במחברת הבחינה

כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:

* ספר הקורס "Essentials of Programming Languages".
* מדריך הלמידה "שפות תכנות".

חומר עזר:

ספר הקורס: essentials of programming languages
מדריך הלמידה.
מותר לכתוב הערות בכתב יד בלבד, רק על גביי מדריך הלמידה.
אסור שימוש בעזרים ובחומרים מקוונים.

אינכם חייבים

להחזיר את השאלון לאוניברסיטה הפתוחה

בהצלחה !!!

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נסיף לשפת "וישגור" יכולת בסיסית של Reflection. Reflection היא היכולת של תוכנית לתחקר בזמן ריצה מרכיבים של התוכנית עצמה לקבל מידע עליהם וביצוע פעולות עליהם. בשאלה זו נרצה לממש יכולת לתיחקור של משתנה בזמן ריצה וקבלת מידע על טיפוסו.

נסיף לשפה מספר ביטויים חדשים.

להלן הדקדוקים של הביטויים החדשים:

Expression ::= get-type (identifier)

get-type-exp (id)

Expression ::= isBool? (Expression)

isBool?-exp (ex)

Expression ::= isNum? (Expression)

isNum?-exp (ex)

Expression ::= isProc? (Expression)

isProc?-exp (ex)

- ביטוי **get-type-exp** מקבל **id** המציין שם של משתנה ומחזיר **expval** המתאר את טיפוסו (שימו לב, זה ביטוי עם סוג חדש של תשובה מוחזרת) הביטוי יבדוק מה מייצג המשתנה (מספר, בוליאני, פרוצדורה) ויחזיר ייצוג של טיפוס מתאים.
- הביטויים **isBool?-exp**, **isNum?-exp**, **isProc?-exp** מקבלים ביטוי **ex**, שאמור להיות מסוג **get-type-exp** ובודקים האם תוצאתו (שהיא תשובה המייצגת טיפוס) היא בוליאנית, מספרית או פרוצדורה בהתאמה. הביטויים מחזירים תשובה בוליאנית (**bool-val**) מתאימה. (המשך השאלה בעמוד הבא)

להלן דוגמה להמחשת היכולות החדשות של reflection :

דוגמה:

```
> (run "let a= 10
  in
    let p= proc (w) - (w,a)
    in
      let q= proc (something)
        let t= get-type(something)
        in
          if isNum?(t)
          then
            (p something)
          else
            if isBool?(t)
            then
              if something then 500 else 800
            else
              (something 50)
        in
          (q zero?(a))")
(num-val 800)
>
```

בדוגמה זו מוגדרת פרוצדורה בשם q המקבלת פרמטר כלשהו בשם something, בגוף הפרוצדורה מתבצע תיחקור בזמן ריצה לגבי טיפוסו של something באמצעות היכולות החדשות שהתווספו לשפה. t מכיל תוצאה המתארת את טיפוסו של something, ולאחר מכן נבדק הטיפוס t מול מגוון טיפוסים שונים, ולפיהם הפרוצדורה פועלת בהתאם.

בהמשך מופעלת הפרוצדורה q עם פרמטר בוליאני שערכו #f, מה שגורם במהלך ביצועה של q ותיחקור הפרמטר שהגיע אליה לכך שיוחזר 800

(המשך השאלה בעמוד הבא)

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "וישגור" (שפת PROC).
 בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בחלקי המפרש, ומהם השינויים והתוספות.
לא תתקבל תשובה של תיאור מילולי. יש לממש בקוד את השינויים הדרושים.

הנחיות כלליות לפתרון:

- (1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- (2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- (3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- (4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- (5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- (6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור (}) או פתוח ({)

שאלה 2 (40 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

בשאלה זו נשנה את האופן בו פרוצדורות מוגדרות ומופעלות, ונאפשר למתכנת לבצע העמסה (overloading) לפרוצדורה שכבר הוגדרה, כך שבהפעלת הפרוצדורה תופעל הגרסה המתאימה שלה במידה וקיימת, בהתאם לסוג הפרמטר הנשלח.

לשם כך יש צורך בהוספת ביטוי חדש לביצוע ההעמסה, שינוי ביטוי `proc-exp` הקיים בשפה להגדרת פרוצדורה, ועדכון תהליך ההפעלה של פרוצדורות.

להלן הדקדוקים המגדירים את ביטוי `proc-exp` המעודכן, וכן את הביטוי החדש להעמסה:

Expression ::= proc (type identifier) Expression

`proc-exp (typ var body)`

type ::= int

`int-type ()`

type ::= bool

`bool-type ()`

type ::= func

`func-type ()`

Expression ::= overload identifier with (type identifier) Expression

`overload-exp (p-name typ var body)`

(המשך השאלה בעמוד הבא)

הסבר הדקדוקים וההתנהגותם:

- ביטוי `proc-exp` להגדרת פרוצדורה עודכן והוא מכיל גם טיפוס לשם הפרמטר של הפרוצדורה כחלק מהגדרתה.
- ביטוי `overload-exp` החדש מקבל שם של פרוצדורה שכבר הוגדרה ומאפשר לשמור גרסה נוספת וחדשה שלה, הגרסה הנוספת החדשה מקבלת פרמטר `var` מטיפוס `typ` ומבצעת את מה ש-`body` מייצג. ניתן להעמיס פרוצדורה מספר פעמים ללא הגבלה.
 - אם ביטוי `overload-exp` מנסה לבצע העמסה של משתנה שאינו מייצג פרוצדורה כלל – תיזרק שגיאה מתאימה על כך.
 - אם ביטוי `overload-exp` מנסה לבצע העמסה של משתנה שלא קיים כלל – תיזרק שגיאה.
 - אם ביטוי `overload-exp` מבצע העמסה לפרוצדורה קיימת עם גרסה נוספת בעלת פרמטר מסוג שונה, הגרסה הנוספת תישמר בנוסף לגרסה הקיימת, וכאשר תהיה דרישה להפעיל את הפרוצדורה עם פרמטר מסוג מסוים תופעל הגרסה המתאימה של הפרוצדורה, במידה וקיימת עבור סוג הפרמטר, אחרת תוחזר שגיאה מתאימה.
 - אם ביטוי `overload-exp` מבצע העמסה לפרוצדורה קיימת עם גרסה נוספת בעלת פרמטר מסוג שכבר קיימת עבורו גרסה קודמת, אזי הפרוצדורה תועמס עם הגרסה החדשה, והגרסה הקודמת לא תהיה רלוונטית יותר.

להלן מספר דוגמאות להמחשת השימוש וההתנהגות של היכולות החדשות המבוקשות:

דוגמה 1:

```
> (run "let sum = 0
    in
      begin
        overload sum with (int b) -(b,50);
        (sum 100)
      end")
ⓧ ⓧ interproc.scm:94:23: overload-exp: cannot overload non procedure sum
```

בדוגמה זו יש ניסיון להעמיס את `sum` שאינו מייצג כלל פרוצדורה, ועל כן התקבלה שגיאה.

דוגמה 2:

```
> (run "let sum = 0
    in
      begin
        overload q with (int b) -(b,50);
        (q 100)
      end")
ⓧ ⓧ environments.scm:30:10: apply-env: No binding for q
>
```

בדוגמה זו יש ניסיון להעמיס משתנה שלא קיים כלל, ולכן התקבלה שגיאה.

(המשך השאלה בעמוד הבא)


```

> (run "let sum = 0
    in
      let p= proc (int a) -(a,10)
      in
        begin
          set sum = -(sum, -(0, (p 100)));
          overload p with (bool b) if b then 700 else 1000;
          set sum = -(sum, -(0, (p zero?(5))));
          overload p with (int r) -(r,300);
          set sum = -(sum, -(0, (p 100)));
          overload p with (func f) (f 30);
          set sum = -(sum, -(0, (p proc (int w) -(w,50))));
          sum
        end")
(num-val 870)

```

בדוגמה זו, מוגדר משתנה `sum` עם ערך התחלתי של 0. וכן פרוצדורה בשם `p` שכרגע יודעת לקבל פרמטר `int` ולהחזיר אותו מופחת ב-10. לאחר מכן מבוצעות מספר העמסות לפרוצדורה `p` וכן עדכון של משתנה `sum`.

בתחילה ערכו של `sum` מתעדכן, ומתווספת לו התוצאה המוחזרת מהפעלת `p` על 100, מאחר ויש ל-`p` גרסה היודעת לקבל פרמטר `int`, הופעלה גרסה זו שהחזירה 90 (הפחיתה 10 מ-100). ולכן ערכו הנוכחי של `sum` הוא 90. לאחר מכן, הפרוצדורה הועמסה עם גרסה היודעת לקבל פרמטר בוליאני, ושוב עודכן ערכו של `sum` והתווספה לו תוצאת קריאה ל-`p` עם פרמטר בוליאני, שהחזירה 1000, ולכן ערכו של `sum` כעת הוא 1090. לאחר מכן, `p` הועמסה שוב עם גרסה היודעת לקבל פרמטר `int`, ומאחר וכבר ישנה גרסה ישנה כזו, הגרסה הישנה לא תקפה יותר. כעת שוב מתבצעת קריאה ל-`p` עם פרמטר מספרי של 100, והפעם מופעלת הגרסה העדכנית עם פרמטר מספרי, המחזירה -200, המתווסף ל-`sum`, ולכן ערכו של `sum` כרגע הוא 890. לבסוף `p` מועמסת עם גרסה היודעת לקבל פרמטר מסוג פרוצדורה, ולאחר מכן, גרסה זו מופעלת ומחזירה במקרה זה, -20, המתווסף לערכו של `sum`, ולכן מצבו הנוכחי של `sum` הוא 870. והוא מוחזר כתשובה סופית של כל התוכנית.

המשך הדוגמאות בעמוד הבא


```
> (run "let sum = 0
  in
    let p= proc (int a) -(a,10)
    in
      (p zero?(5))")
@ @ interp.scm:145:10: apply-procedure: No procedure found with #(struct:bool-type) argument
>
```

בדוגמה זו ישנו ניסיון להפעיל פרוצדורה p שאין עבורה גרסה עם פרמטר בוליאני, ולכן התקבלה שגיאה מתאימה.

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וירמוז" (שפת IMPLICIT-REFS).

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}*) או פתוח ({+})

שאלה 3 (25 נקודות)

שימו לב – זוהי שאלה מפוצלת ל-2 גרסאות שונות. עליכם לפתור את הגרסה המתאימה לסמסטר הלימוד בו למדתם את הקורס. פתרון של גרסה שגויה לא יזכה בניקוד

גרסה עבור סטודנטים שלמדו בסמסטר 2024ב:

להלן תוכנית בשפת PROC (שפת "וישגור")

```
let p = proc (a) (a 6)
```

```
in
```

```
(p proc (b) zero?(b))
```

רשמו את שלבי חישוב התוכנית ע"י המפרש תחת סביבה התחלתית ריקה, באמצעות שימוש בסימונים מוסכמים כמתואר למשל בספר הלימוד בעמודים 77-78.

לבסוף רשמו מהי תוצאת התוכנית?

גרסה עבור יתר הנבחנים:

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = proc (a:?) (a 6)
```

```
in
```

```
(p proc (b:int) zero?(b))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש באלגוריתם שנלמד בפרק 7 בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה !